

EMBEDDED SYSTEMS HACKATHON

Modular **Vehicle ECU** Simulation

Design and implementation of a deterministic, cyclic-execution Embedded C application for automotive control logic and fault management.

System Objectives

Modular Architecture

Implement a structured design with clear separation of concerns using separate .c and .h modules for inputs, logic, and state management.

Deterministic Timing

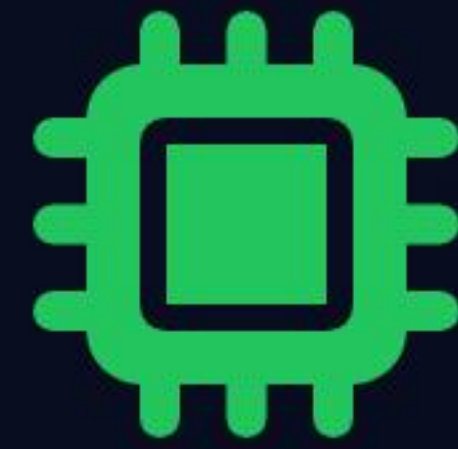
Enforce a cyclic execution model where tasks run in a fixed order, mimicking real-world automotive software schedulers.

Defensive Logic

Integrate robust input validation and fault tracking to ensure system stability under invalid operating conditions or transitions.

State Management

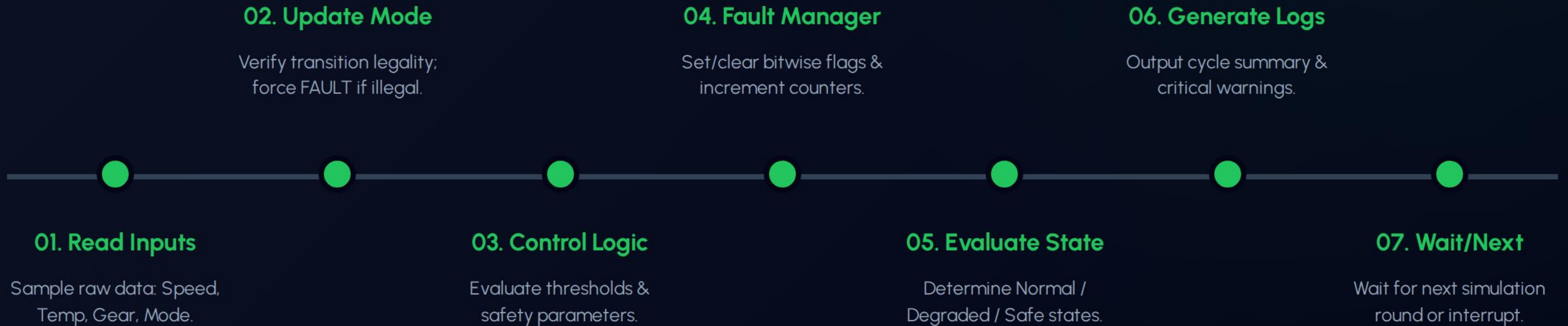
Maintain deterministic state transitions (Normal/Degraded/Safe) based on real-time sensor feedback and persistent fault counters.



Cyclic Scheduler Logic

THE CORE EXECUTION FLOW

Cycle Execution Flow



Input Layer Constraints

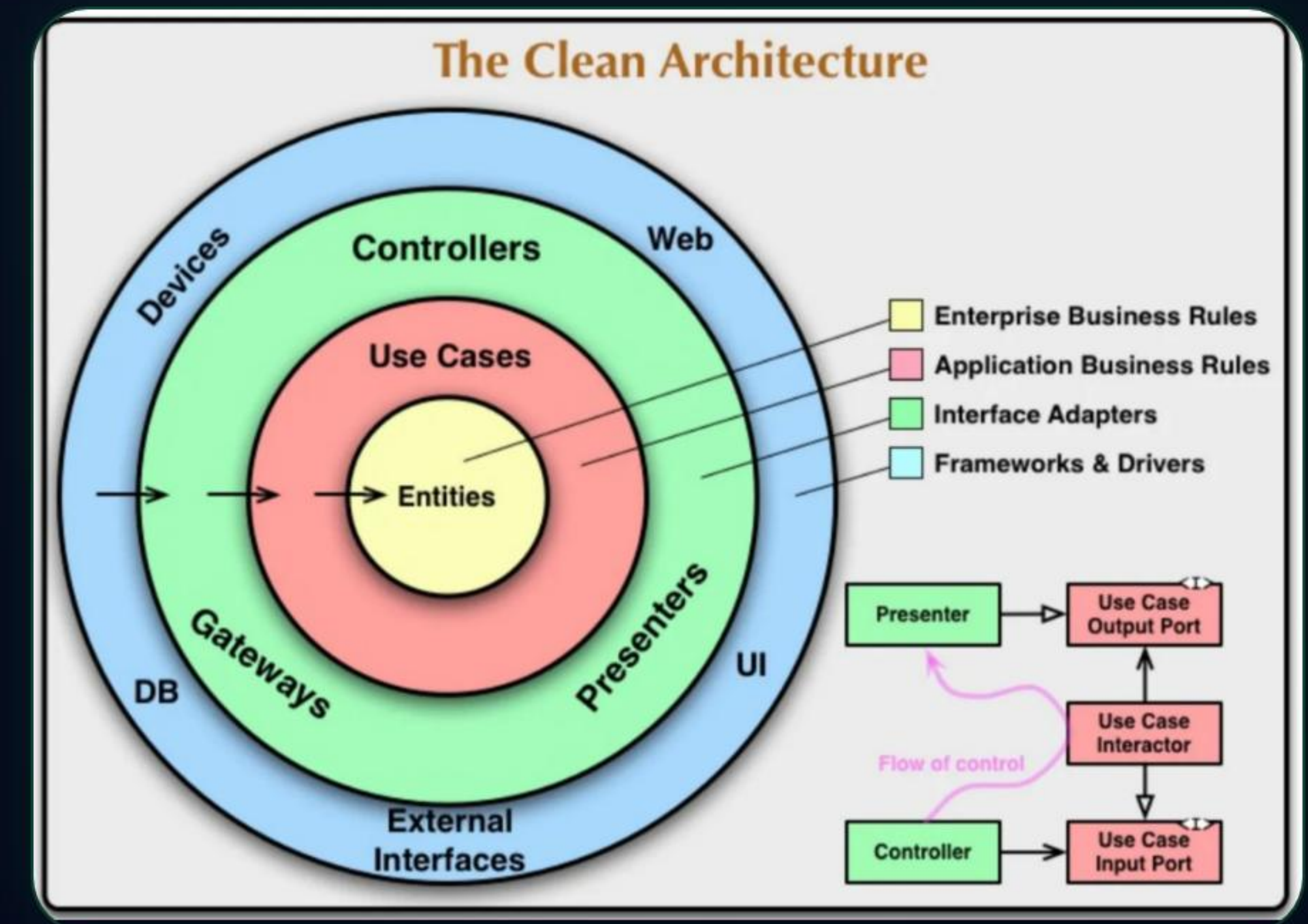
Parameter	Valid Range	Unit / Type	Failure Behavior
Vehicle Speed	0 to 200	km/h	Log error; preserve last valid value.
Engine Temperature	-40 to 150	°C	Log error; apply default safe value.
Gear Position	0 to 5	Integer	Raise fault flag; increment counter.
Operating Mode	Defined Enum	Enum	Transition to FAULT mode immediately.

Finite State **Machine**

Mode Logic Requirements

The Mode Controller manages system lifecycle using an Enum-based Finite State Machine (FSM). It strictly enforces legal transitions to prevent engine damage or unsafe operation.

- ✓ **Legal:** OFF → ACC → IGNITION_ON
- ⊘ **Illegal:** OFF → IGNITION_ON (Direct jump)
- ⚠ **Safety:** Invalid transitions force FAULT mode



Critical Safety Thresholds



Thresholds trigger hierarchical fault reporting based on defined priorities: Critical Overheat > Invalid Mode > Overspeed.

Fault Management Engine



Bitwise Flags

Efficiently store multiple active faults in a single integer using bitwise positions (e.g., BIT_0: Speed, BIT_1: Temp).



Persistence

Track faults across multiple cycles. A fault remains "set" until specific recovery logic or reset conditions are met.



Fault Counters

Every occurrence increments a dedicated counter. Persistent failures escalate system state from NORMAL to SAFE.

Safe-State Escalation

2+

Critical Faults

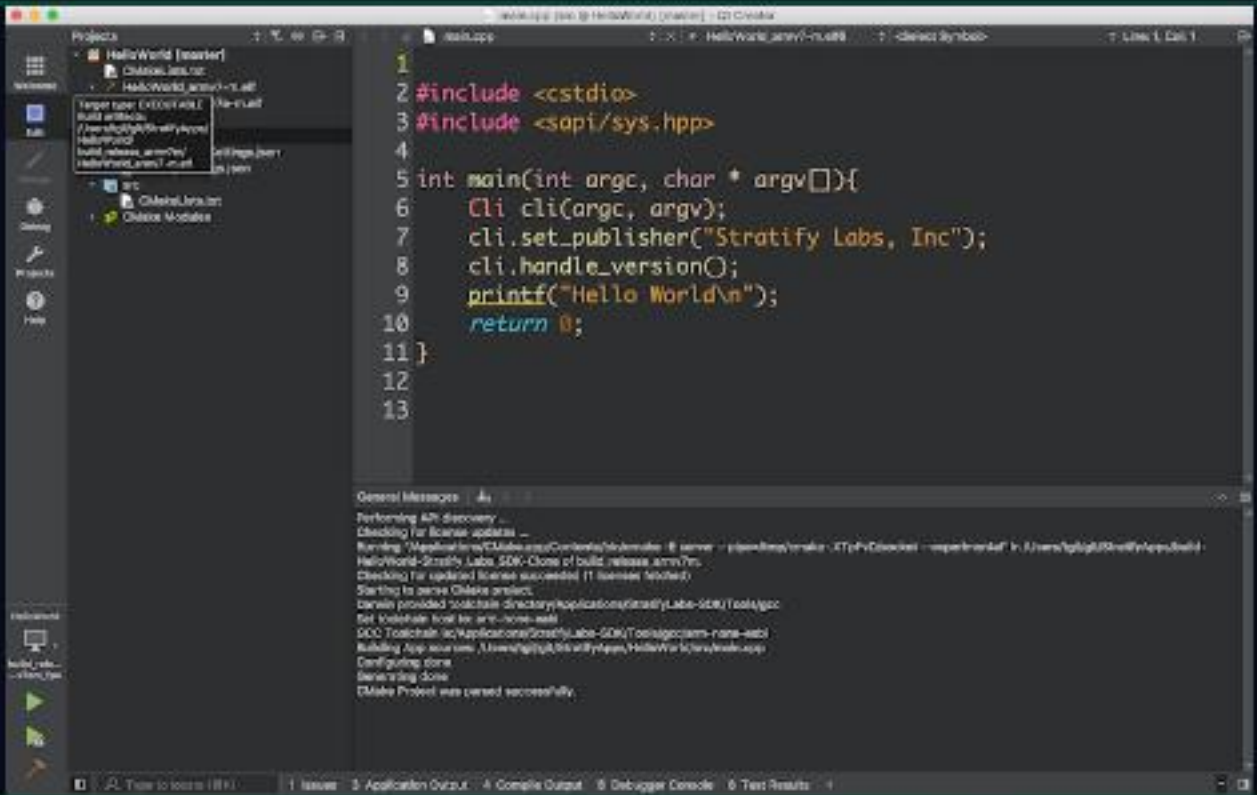
Transition to SAFE State

The ECU employs a multi-tiered safety strategy to protect vehicle hardware. State changes are deterministic and logged with a specific reason.

- 🛡️ **NORMAL:** No active faults; standard operation.
- ⚠️ **DEGRADED:** One major fault or repeated warnings.
- 🔌 **SAFE:** Two or more persistent critical faults. System locked.

Modular File Structure

<code>main.c</code>	Scheduler loop & Integration	<code>input.c / .h</code>	Input validation & Scaling
<code>mode.c / .h</code>	FSM & Mode Transitions	<code>control.c / .h</code>	Logic checks & Thresholds
<code>fault.c / .h</code>	Bitwise flags & Counters	<code>state.c / .h</code>	Safety escalation logic
<code>log.c / .h</code>	Structured console output	<code>types.h</code>	Global Enums, Structs, Macros



Validation Matrix

- 🔧 **Normal Start:** OFF → ACC → IGN
- 🔧 **Overspeed:** Speed 130 triggers counter
- 🔧 **Overheat:** Temp 115 triggers SAFE state
- 🔧 **Invalid Gear:** Out-of-range value fault
- 🔧 **Illegal Transition:** OFF → IGN directly
- 🔧 **Persistent Fault:** Multi-cycle escalation



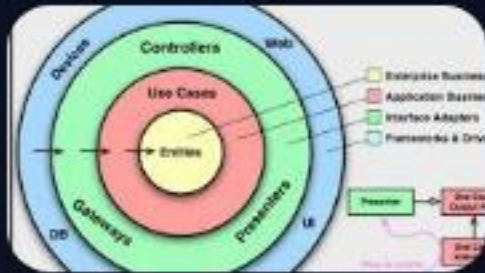
System Ready

MODULAR ECU SIMULATION FINALIZED

Deterministic cyclic execution achieved with MISRA-compliant defensive patterns. Project deliverables include modular Source, Header guards, and full Test Scenarios.

Questions? | Embedded C Application Development

Image Sources



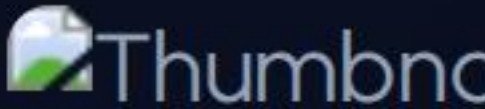
<https://storage.googleapis.com/bitloops-github-assets/Documentation%20Images/Clean-Architecture-Uncle-Bob.png>

Source: bitloops.com



Thumbnail for <https://blog.stratifylabs.dev/images/qt-creator-screen-shot.png>

blog.stratifylabs.dev Source: blog.stratifylabs.dev



Thumbnail for <https://cdn.shopify.com/s/files/1/0241/4313/4816/files/cluster-warning-lights-explained.jpg?v=1698169613>

issautomotive.com Source: issautomotive.com